

DDLaunch+ (Data-Driven Launcher+)

v2.3.0

A small, native Win32 C++ desktop launcher. It lists remote resources (hosts, drives, shell items, URLs) from a JSON file and opens each one with a tool chosen by its **type**. Which tool handles which type is also defined in JSON, so you can add new connection types without recompiling.

Author	Glenn Madine
Version	2.3.0 (release date 09/26/2026)
Language / API	C++17, Win32 API, Common Controls v6, COM (Shell.Application)
Compiler	Microsoft C++ 19.51 (MSVC)
Dependencies	Windows SDK, nlohmann/json (single header <code>json.hpp</code>)
Build files	<code>DDLaunch.cpp</code> , <code>DDLaunch.rc</code> , <code>DDLaunch.manifest</code> , <code>resource.h</code> , <code>DDLaunch.ico</code>
Source file	<code>DDLaunch.cpp</code> (~1,870 lines)
Minimum OS	Windows Vista or later; best on Windows 10 1703+ (Per-Monitor V2 DPI)

1. What it does

When started, DDLaunch+ opens a resizable window where it was last closed (or at a default size scaled for the monitor), always kept fully inside the monitor's visible area. The window contains:

- **A report-style list view** with three sortable columns: *Resource (Host, Drive, or Shell Item)*, *Type*, and *Description*.
- **Five buttons** in two rows, sized to fit their captions:

Button	Behavior
Launch RDP connection for host not in this list	Starts <code>mstsc.exe</code> with no arguments; reports an error if it can't start
Edit actionDefinitions.json	Opens the actions file in Notepad
Edit connections.json	Opens the connections file in Notepad
Run...	Shows the standard Windows Run dialog (same as Win+R)
Exit	Closes the application

Launching. Double-click a row, or press **Enter** on it. The app looks up the row's **Type** in `actionDefinitions.json`, ignoring upper/lower case (`ssh`, `Ssh` and `SSH` all match):

- If the action's `command` starts with a URI scheme (`https://`, `file:///`, `ssh://`, `vnc://`, `mailto:`, `ms-settings:...`), the app opens `command + host` with the program registered for that scheme.

- Otherwise the app runs `command` as a program, with `args + host` as its parameters. A host that contains spaces, tabs or quotes is quoted using the Windows command-line rules, so it arrives as one argument (for example `/v: "Lab PC 7"`).

Error messages. The app shows a specific message when:

- the entry has no type
- no action matches the type (the message lists the actions that are defined)
- the action's command is empty
- the launch fails (the message includes the Windows error code and text)
- `actionDefinitions.json` contains duplicate action names (shown once when the file loads; only the first of each name is used)

Sorting. Click a column header to sort by that column, ignoring case. Click it again to reverse the order. The sort, and the arrow in the header, are kept when the data reloads. The selected row stays selected (and scrolled into view) across sorts and reloads.

Hot reload. When you switch back to the window, it reloads a JSON file only if the file's last-write time or size has changed. Edits made in Notepad show up right away, and an invalid file reports its error once. While either file is invalid, the app keeps the data it last loaded successfully, so the list never goes blank because of a typo.

Single instance. Only one copy runs per Windows sign-in session. Starting DDLaunch+ again brings the existing window to the front, restoring it if it's minimized. Separate users and Remote Desktop sessions each get their own copy.

Window title. "DDLaunch+ v2.3.0 - double-click a row or press Enter to launch".

Keyboard.

Key	Action
Tab / Shift+Tab	Move between the list and the buttons
Arrow keys	Move within the list
Enter	Launch the selected row, or press the focused button
Space	Press the focused button
Esc	Ignored on purpose, so it can't close the launcher

When the window is re-activated, focus goes back to the control that last had it.

High DPI. The app runs Per-Monitor V2 DPI aware. The font (the system message font, normally Segoe UI 9pt), column widths, buttons and margins scale with the monitor, and they update live when the window moves to another monitor. The window has a minimum size so the buttons never overlap.

Remembered placement. On exit (including logoff and shutdown), the window's position, size and maximized state are saved to `settings.json`. At the next start the window reopens there. The size is stored in 96-DPI units, so it looks the same on monitors with different scaling. If that monitor is gone, the window moves to the nearest one and is clamped to its work area.

2. Configuration files

All files live in a per-user folder:

```
%USERPROFILE%\DDLlaunch\actionDefinitions.json
%USERPROFILE%\DDLlaunch\connections.json
%USERPROFILE%\DDLlaunch\settings.json      (window placement, written on exit)
```

If `%USERPROFILE%` is not set, the app uses the shell's Profile known folder instead. If that also fails, it uses the folder that contains `DDLlaunch.exe`. The folder and the two JSON config files are created on first run; `settings.json` is created the first time the app exits. Existing config files are never overwritten.

2.1 actionDefinitions.json

The file must be a top-level JSON **array**. Every entry needs all three keys, or the file fails to load (the app then keeps the actions it last loaded successfully). Action names are matched without regard to upper/lower case. If two actions differ only by case, the first one in the file is used, and the app warns about the duplicate when the file loads.

Key	Meaning
action	Name matched against a connection's <code>type</code> , ignoring case (e.g. <code>RDP</code> matches <code>rdp</code>)
args	Text placed before the host on the command line (e.g. <code>" /v:"</code>). Not used for URL commands
command	Path to a program, or a URL prefix with any scheme (e.g. <code>https://</code> , <code>vnc://</code>)

Default content written on first run:

```
[
  { "action": "RDP",    "args": " /v:", "command": "C:\\Windows\\System32\\mstsc.exe" },
  { "action": "SSH",    "args": " ",   "command": "C:\\Windows\\System32\\OpenSSH\\ssh.exe" },
  { "action": "HTTPS",  "args": "",    "command": "https://" },
  { "action": "HTTP",   "args": "",    "command": "http://" }
]
```

Example of a custom action, which needs no recompile:

```
{ "action": "VNC", "args": "", "command": "vnc://" }
```

How the app tells a URL from a program path: it looks for a scheme as defined by RFC 3986 (ALPHA *(ALPHA / DIGIT / "+" / "-" / ".") ":"). Schemes must be at least two characters long, so drive letters such as `C:\...` are still treated as program paths.

2.2 connections.json

The file can be a flat array of device objects, or an object of named groups (each group is an object or an array of objects). Each device can have `host`, `type`, and `description` (all strings, all optional). The default is an empty array `[]`.

```
[
  { "host": "dc01.ynext.corp", "type": "RDP", "description": "Domain controller" },
  { "host": "piweb.ynext.corp", "type": "HTTPS", "description": "Raspberry Pi web server" },
  { "host": "admin@pi01", "type": "ssh", "description": "Pi shell (type is case-insensitive)" }
]
```

Grouped form (group names are not shown; all devices are merged into one list):

```
{
  "Servers": [ { "host": "ca1.ynext.corp", "type": "RDP", "description": "Certificate Authority" } ],
  "Web":      { "host": "intranet.ynext.corp", "type": "HTTPS", "description": "Intranet" }
}
```

2.3 settings.json

The app manages this file itself; you don't need to edit it.

```
{
  "window": { "x": 120, "y": 80, "width": 640, "height": 420, "maximized": false }
}
```

- `x` and `y` are screen coordinates in pixels.
- `width` and `height` are in 96-DPI units.
- The file is written to a temporary file first and then swapped in, so a crash mid-write can't leave it half-written.
- If the file is missing, corrupt, or has implausible values (a size under 100 or over 20,000), the app uses its defaults without showing an error.

3. Building

Requirements: Visual Studio (MSVC 19.x) with the Windows SDK, `json.hpp` from `nlohmann/json`, and `resource.h` (defines `IDI_ICON1`). You also need the resource script `DDLaunch.rc`, which contains the icon, the manifest and version info, and `DDLaunch.manifest`.

From a *Developer Command Prompt for VS*:

```
rc /nologo DDLaunch.rc
CL /EHsc /W3 /O2 /GL /DUNICODE /D_UNICODE /DNDEBUG /std:c++17 DDLaunch.cpp DDLaunch.res /Fe:DDLaunch.exe
/link /SUBSYSTEM:WINDOWS /MANIFEST:NO comctl32.lib shell32.lib shlwapi.lib user32.lib gdi32.lib ole32.lib
oleaut32.lib
```

Manifest (single source). `DDLaunch.manifest` is the only manifest for the `.exe`. `DDLaunch.rc` embeds it as `RT_MANIFEST` resource 1, and `/MANIFEST:NO` stops the linker from generating a second one, so duplicate-manifest link errors can't happen. In a Visual Studio project, set *Linker* → *Manifest File* → *Generate Manifest* = *No*. The manifest declares:

Entry	Purpose
Common Controls 6.0.0.0 dependency	Visual styles, header sort arrows, themed buttons

Entry	Purpose
<code>requestedExecutionLevel asInvoker</code>	Never asks to run as administrator; opts out of legacy file and registry virtualization
<code>supportedOS Windows 7, 8, 8.1, 10/11</code>	Turns off compatibility shims
<code>dpiAwareness PerMonitorV2, PerMonitor + dpiAware true/pm</code>	High-DPI mode, with fallbacks for older Windows

DPI. The manifest sets DPI awareness. `Dpi::EnableDpiAwareness()` is only a fallback: it does nothing if the process is already DPI aware.

Build self-check. At startup, `CheckCommonControlsVersion()` checks that Common Controls v6 is loaded. If it isn't (the manifest wasn't embedded), it writes a warning to the debugger output, and debug builds also show a message box.

4. Code architecture

Section	Key functions / types	Purpose
Header / constants	<code>VERSION</code> , <code>IDC_*</code> , <code>COL_*</code> , <code>UNICODE</code> guards	Version, control IDs, column indexes
UTF-8 ↔ UTF-16	<code>Utf8ToWide</code> , <code>WideToUtf8</code> , <code>ExceptionText</code>	Convert between JSON (UTF-8) and Win32 wide strings; exception text shown safely in wide message boxes
Action model	<code>struct Action</code> , <code>from_json</code>	nlohmann/json mapping for action entries (read-only)
File I/O	<code>ReadFileUtf8</code>	Reads a whole file via <code>CreateFileW/ReadFile</code> (supports Unicode paths)
Action lookup	<code>loadActions</code> , <code>findAction</code>	Parse the actions array; a single lookup that ignores case (<code>CompareStringOrdinal</code>) and returns <code>const Action*</code>
Device model	<code>struct Device</code> , <code>LoadConnections</code>	Parse connections (array or grouped object); returns <code>std::nullopt</code> on failure so the old list is kept
Paths	<code>GetUserProfileDir</code> , <code>GetDataDir</code> , <code>Get*JsonPath</code>	Resolve <code>%USERPROFILE%\DDLlaunch</code> , with fallbacks
First-run setup	<code>EnsureDataFiles</code> , <code>CreateFileWithDefaultContent</code>	Create the folder and write the default JSON (<code>CREATE_NEW</code> , safe if two instances race)
Editing	<code>OpenJsonFileForEditing</code>	Opens a config file in Notepad
URL detection	<code>StartsWithUrlScheme</code>	Generic RFC 3986 scheme check (ignores drive letters)
Launching	<code>QuoteArgIfNeeded</code> , <code>ShellOpen</code> , <code>LaunchSomething</code> , <code>ReportLaunchFailure</code> , <code>GetErrorText</code>	Quote hosts that contain spaces or quotes; <code>ShellExecuteExW</code> with <code>SEE_MASK_FLAG_NO_UI</code> ; clear error messages

Section	Key functions / types	Purpose
Run dialog	ShowRunDialog	IShellDispatch::FileRun() via COM
DPI helpers	namespace Dpi	Fallback DPI awareness, get DPI per window and per monitor, scale values, create the UI font, adjust window rect (newer APIs loaded dynamically)
Work-area clamp	ClampToWorkArea	Keep a window rect inside its monitor's visible area (R10)
Sorting / list	ApplyCurrentSort, ToggleSortColumn, UpdateHeaderSortArrow, PopulateListView	Stable sort re-applied after every reload; the list is refilled without flicker and keeps the selected row
Device IDs	Device::id, AssignDeviceIds, FindDeviceIndexById, GetRowDeviceId	A stable ID per connection, stored in the row's lParam; carried over across reloads
Change detection	FileStamp, GetFileStamp, ReloadAllData(hwnd, force), WarnAboutDuplicateActions	Reload a file only when it changed; guarded against re-entry while an error box is open
Item activation	OnItemActivated, ActivateSelection	Find the selected connection by its ID, check its type and action, then launch
Layout	namespace Layout, S(), MeasureButtonWidth, ApplyDpi, CreateButton	Sizes in 96-DPI units, scaled per monitor; buttons sized to fit their text
Placement persistence	WindowSettings, LoadWindowSettings, SaveWindowSettings, TrackNormalRect, RescaleForDpi	Save and restore position, size and maximized state (R8)
Window procedure	WndProc	Create, size, move, min/max size, DPI change, commands, notifications, activation/focus, end-session, save on destroy
Single instance	AcquireSingleInstance, kInstanceMutex, kWindowClass	A named mutex in the Local\ namespace; a second copy activates the first and exits
Build self-check	CheckCommonControlsVersion	Warn if Common Controls v6 isn't loaded (R11)
Entry point	wWinMain	Single-instance check, DPI fallback, common controls, window class, restore and clamp placement, IsDialogMessageW message loop

Launch flow

1. The user double-clicks a row, or presses Enter on it. Enter arrives as `IDOK` via `IsDialogMessageW`, or as `NM_RETURN`. `ActivateSelection` reads the row's device ID and finds the connection with `FindDeviceIndexById`.
2. `OnItemActivated` checks that the entry has a type, that an action exists for it (matched ignoring case), and that the command isn't empty.
3. `LaunchSomething` uses `StartsWithUrlScheme(command)` to choose one of two calls:
 - **URL:** `ShellExecuteExW("open", command + host)`

- **Program:** `ShellExecuteExW("open", command, args + QuoteArgIfNeeded(host))`
- 4. On failure, `ReportLaunchFailure` shows the host, type, command, parameters, and the Windows error code and text. A cancelled UAC prompt is not reported as an error.

Reload flow

1. `WM_CREATE` calls `ReloadAllData(hWnd, true)`, which loads both files.
2. On each `WM_ACTIVATE`, `ReloadAllData(hWnd, false)` compares the current `FileStamp` of each file with the stored one. If both match, it returns straight away.
3. For each file that changed, the new stamp is recorded **before** the file is parsed, so an error is shown once per save.
4. If `connections.json` loads, `AssignDeviceIds` carries each old ID over to the matching new entry (first exact host+type+description, then host+type); new entries get new IDs. Devices are then re-sorted with the current sort and the list is repopulated. If loading fails, the previous devices stay in place.
5. If `actionDefinitions.json` loads, `WarnAboutDuplicateActions` reports any names that are the same ignoring case. If loading fails, the previous actions stay in place.
6. `PopulateListView` reads the selected row's ID before clearing the list, then reselects the row with that ID and scrolls it into view.

Startup placement flow

1. `AcquireSingleInstance()` creates the named mutex. If another copy already owns it, the new process finds that copy's window (waiting up to about 2 seconds while it starts), restores it if minimized, brings it to the front, and exits.
2. The window is created hidden with `CW_USEDEFAULT`.
3. `LoadWindowSettings()` reads the saved placement, if there is one:
 - **Saved placement:** find the monitor nearest the saved rectangle, get its DPI (`GetDpiForMonitor`), and scale the saved 96-DPI size to that DPI.
 - **No saved placement:** use the position Windows chose, with the default 600 × 400 size scaled for that monitor.
4. `ClampToWorkArea()` shrinks and shifts the rectangle so it fits inside the monitor's work area.
5. `SetWindowPos` is called while `g_placingWindow` is set, so a `WM_DPICHANGED` along the way doesn't scale the rectangle a second time. Then `RescaleForDpi` updates the fonts, columns and buttons for wherever the window ended up.
6. The window is shown maximized if it was closed maximized and the shortcut asked for a normal window; otherwise it follows the shortcut's setting.

Strengths

- **Unicode throughout:** wide Win32 APIs, UTF-8 JSON, and Win32 file I/O, so profile paths with non-ASCII names work.
- **Data-driven:** a new launch type (including a custom URL scheme) needs only a JSON edit.
- **Clear feedback:** every failure path produces a specific message that says what to change and where.
- **Reload cost:** it happens only when a file changes, and it doesn't loop on errors.
- **Keyboard:** full keyboard operation, with tab order matching the layout and focus restored on activation.

- **High DPI:** Per-Monitor V2 with live rescaling, fallbacks for older Windows versions, and a minimum window size.
- **Documented APIs only:** uses `Shell.Application` for the Run dialog, and loads the newer DPI APIs dynamically so the .exe still starts on older systems.
- **Remembers its window:** placement is saved safely, restored at the right size on any DPI, and always opens on-screen.
- **Robust selection and launching:** connections are tracked by a stable ID, not by row position or cell text, and hosts with spaces are quoted correctly.
- **One copy at a time:** a second launch activates the running window, so there are no competing `settings.json` writes.
- **Resilient to bad edits:** a broken JSON file never empties the list or loses actions, and error text displays correctly for any profile name.
- **Clean code:** no dead code or unused includes; the build has no warnings under GCC `-Wall -Wextra -Wunused`. Single-source manifest and explicit includes.
- **Maintainable:** a version-history comment block in the header records each change.

5. Analysis: remaining issues and recommendations

Severity: **Medium** = incorrect or fragile behavior; **Low** = cleanup, maintainability, or minor UX.

5.1 Behavior

#	Severity	Issue	Recommendation
B12	Low	Entries without a host are still listed. They show as blank rows and only fail on launch.	Skip them when loading, or show them with a warning marker.
B15	Low	URL hosts are not encoded. For URL commands the host is appended as-is, so a host containing spaces or <code>#</code> , <code>?</code> , <code>%</code> produces a malformed URL. Real host names rarely contain these characters.	Percent-encode the host (e.g. <code>UrlEscapeW</code> with <code>URL_ESCAPE_SEGMENT_ONLY</code>) or reject such hosts with a clear message.

Fixed since the previous review: **B11** (hosts with spaces are quoted), **B13** (duplicate action names are reported), and **B14** (rows are tracked by a stable ID instead of cell text).

5.2 Dead or duplicated code

None remaining. No dead code was added in 2.3.0; GCC `-Wall -Wextra -Wunused` still reports no warnings. The items removed in 2.2.0:

#	Item	Resolution
D1	<code>UsernameDialog</code> , <code>GetUsernamePopup()</code>	Removed
D2	<code>class Logger</code>	Removed
D3	<code>getExeDirectory()</code>	Removed; <code>GetExeDir()</code> remains as the last-resort data folder

#	Item	Resolution
D4	<code>ToWide()</code> , <code>WStringToString()</code>	Removed; all calls use <code>Utf8ToWide()</code> / <code>WideToUtf8()</code>
D6	Unused <code>actions</code> parameter of <code>PopulateListView</code>	Removed
D7	<code>to_json(Action)</code>	Removed (the app never writes actions)
D8	<code>#include <fstream></code> , <code><sstream></code>	Removed

5.3 Build and robustness notes

#	Item	Recommendation
R13	Snapped windows	An Aero Snap layout (half or quarter screen) is recorded as the normal size, so the next start reopens at the snapped size but not snapped. Acceptable, but worth knowing.
R15	Single-instance edge cases	If the first copy is still starting up and showing an error box, or is hung, the second copy has nothing visible to activate and exits quietly. If the second copy wasn't started by the user (e.g. from a scheduled task), Windows may only flash the taskbar button instead of switching windows. Both are rare; a registered "show yourself" message, plus <code>AllowSetForegroundWindow</code> , would make it fully robust.
R14	Manifest is now required at build time	With <code>/MANIFEST:NO</code> , a build that leaves out <code>DDLlaunch.res</code> has no manifest at all (no visual styles; system-DPI fallback only). <code>CheckCommonControlsVersion()</code> detects this; keep <code>DDLlaunch.rc</code> in the project.

Fixed in earlier releases: **R1**, **R2**, **R4**, **R6**, **R8**, **R10** and **R11**. Fixed in 2.3.0: **R5** (the stale comment), **R9** (the window title) and **R12** (single instance).

5.4 Security note

DDLlaunch+ runs whatever `command` the JSON says, with the current user's rights. That is by design, and the files sit in the user's own profile, so the risk is the same as a user shortcut. Don't point DDLaunch+ at config files in shared or writable-by-others locations.

6. Version history

Version	Changes
2.3.0	B11: hosts with spaces or quotes are quoted using Windows command-line rules (<code>QuoteArgIfNeeded</code>); B13: duplicate action names are reported on load; B14: stable device IDs in <code>lParam</code> for selection and launching (carried across reloads); R12: single instance (named mutex, activates the existing window); R9: new window title; R5: stale comment fixed
2.2.0	B5: case-insensitive action lookup (<code>CompareStringOrdinal</code>); B9: selection kept across sorts and reloads, flicker-free refill; B8: the list is kept when <code>connections.json</code> fails to load; B10: wide message boxes for JSON errors (<code>ExceptionText</code>); dead code and unused includes removed (D1-D4, D6-D8)

Version	Changes
2.1.2	R11: single-source manifest (DDLlaunch.manifest embedded by DDLlaunch.rc, /MANIFEST:NO, pragma removed); manifest adds asInvoker, supportedOS and PerMonitorV2; the DPI fallback respects the manifest; startup check for Common Controls v6
2.1.1	R1: UNICODE guards; R2: explicit <cwctype>/<cstdint>; R8: window position, size and maximized state saved to settings.json (DPI-independent, atomic write, also on logoff); R10: startup rectangle clamped to the monitor work area
2.1.0	Keyboard navigation (IsDialogMessageW, tab order, focus restore, Enter launches); Common Controls v6 manifest dependency; Per-Monitor V2 DPI awareness with live rescaling; buttons sized to their text; minimum window size (fixes R4); sharper icons
2.0.3	B3: clear messages for a missing type, an unknown type, a case-mismatched action name, an empty command, and failed launches (ShellExecuteExW + Win32 error text); single findAction() lookup
2.0.2	B4: generic RFC 3986 URL-scheme detection replaces the hard-coded list; fixes file:///
2.0.1	B1: reload only when a file's timestamp or size changes (no error-box loop, single load at startup); B2: sort order re-applied after reload (stable sort)
2.0.0	Initial documented release

7. Suggested next steps

1. Skip or flag connection entries that have no host (B12).
2. Percent-encode hosts for URL commands, or reject unsafe ones (B15).
3. Make the single-instance handoff fully robust with a registered "show yourself" message and AllowSetForegroundWindow (R15).
4. Optional enhancements: a filter box above the list for large connection files, and a right-click menu (Copy host, Launch, Edit connections.json).

Documentation generated 09/26/2026 from DDLaunch.cpp v2.3.0.